Made with GAMMA

# Travel Booking Management System

## A Dynamic Linked List Approach to Passenger Records

Made with GAMMA

# The Problem

## The Problem

Traditional booking systems face data redundancy and real-time update challenges. Fixed-size arrays cause memory wastage or overflow when bookings exceed capacity.

> ⓘ A system that dynamically grows and shrinks based on passenger volume — with quick search and deletion — is essential.

## Real-World Applications

- ✈️ Airline Reservation - Managing passenger manifests for specific flights
- 🚌 Bus / Train Ticketing - Real-time seat allocation and cancellation management
- 🏨 Hotel Management - Guest check-ins and room assignment tracking
- 🎟️ Event Ticketing - High-volume ticket sales with instant add/remove

Made with GAMMA

# Data Structure: Singly Linked List

### Dynamic Memory

Uses only as much RAM as needed — no pre-allocation waste

### Efficient Operations

Insertions and deletions require no element shifting

### Custom Struct

Mixed data types (int, char, float) with a self-referential *next pointer

## Node Structure

```
struct booking {
    int bookingID;
    char name[50];
    char destination[50];
    int seats;
    float price;
    struct booking *next;
};
```

## Functional Modules (CRUD)

- Create: addBooking()
- Read: displayBookings() & searchBooking()
- Update: updateBooking()
- Delete: deleteBooking()

Made with GAMMA

# Insertion Algorithm

01

Allocate Memory - Create a newNode using malloc() and input passenger details

02

Set Pointer - Set newNode->next = NULL

03

Attach to List - If head == NULL, assign newNode as head; else traverse to last node and link

Made with GAMMA

# Deletion Algorithm

01

Input Target ID - Read the bookingID to delete; if list is empty, display "No Records Found"

02

Check Head Node - If head contains the target ID: move head to head->next, then free() old head

03

Traverse & Unlink - Track prev and curr; when found: prev->next = curr->next, then free(curr)

Made with GAMMA

# Demo Output

## Adding & Displaying Records

```
1. Add Booking
Enter Booking ID: 101
Enter Passenger Name: Alice
Enter Destination: London
Enter Seats: 2
Enter Price: 450.00
Booking Added Successfully!

--- Booking Records ---
Booking ID : 101
Name       : Alice
Destination: London
Seats      : 2
Price      : £450.00
```

## Searching & Deleting Records

```
4. Search Booking
Enter ID: 101
Booking Found!
Name       : Alice
Destination: London

5. Delete Booking
Enter ID: 101
Booking Deleted Successfully!

--- Booking Records ---
(No records found)
```

Made with GAMMA

# Future Enhancements

### 💾 Data Persistence

Save records to a .csv or .dat file so data survives application exit

### 🔃 Sorting

Implement sorting algorithms to order bookings by ID or passenger name

### 🔍 Search Optimisation

Adopt a Doubly Linked List for two-way traversal and faster lookups

### 🖥️ GUI Interface

Migrate from console-based interaction to a modern graphical user interface

Made with GAMMA

# Conclusion

## Conclusion

The Travel Booking Management System successfully demonstrates the power of dynamic data structures in a practical, real-world context.

### By utilising Singly Linked Lists, the system remains:

- Memory-efficient — allocating only what is needed
- Highly responsive — instant insertions and deletions
- Scalable — a foundational architecture for complex reservation engines

This project lays the groundwork for production-grade booking platforms built on dynamic, pointer-based data structures.

Questions?

The system's robust design provides a stable and efficient base for future growth and complex requirements in travel booking and other real-time data management applications.

Made with GAMMA